
CSE 256 WI 26 — PA2 Report: Transformer Blocks

Harmin Qi
UC San Diego
h5qi@ucsd.edu

1 Part 1: Encoder Trained with Classifier

1.1 Model and training setup

I implemented a Transformer encoder from scratch and trained it end-to-end with a simple feedforward classifier for 3-way speech segment classification (Obama / G.W. Bush / G.H. Bush), following the PA2 base setting. The encoder uses absolute token embeddings plus absolute positional embeddings. Each encoder block consists of multi-head self-attention followed by a two-layer feedforward network with ReLU activation, with residual connections and LayerNorm.

To feed the encoder outputs into the classifier, I used mean pooling over the sequence dimension, masked to exclude padding tokens, which matches the assignment specification for the base setting. I then applied a one-hidden-layer MLP classifier (hidden dimension 100) to predict the politician label.

The total number of encoder parameters is 550,032.

1.2 Sanity checks: attention normalization and visualization

I used the provided sanity-check procedure to verify attention map normalization and to visualize attention patterns. Concretely, I checked that attention probabilities sum to 1 over each query row (within numerical tolerance) and then rendered attention heatmaps. I include attention heatmaps for two sentences and four layers (8 panels total) in Figure 1. The plots are consistent with a valid normalized attention distribution: probabilities concentrate on a subset of tokens rather than spreading uniformly.

Qualitatively, the attention structure differs across layers, suggesting progressive refinement of token interactions. Earlier layers tend to distribute attention more broadly (capturing short-range co-occurrence), while later layers exhibit sharper peaks (capturing more selective dependencies). This aligns with the common interpretation that deeper layers in Transformers can encode more abstract contextual interactions.

1.3 Results: test accuracy per epoch

Table 1 reports train/test accuracy after each of the 15 epochs, as required. The final test accuracy is 84.93%, which falls in the expected low-to-mid 80s range.

The training accuracy increases rapidly and approaches 99%, while test accuracy saturates in the mid-80s. This indicates a growing generalization gap, suggesting that regularization and/or early stopping could further stabilize performance. In this assignment, I keep the base setting and report results as obtained.

2 Part 2: Pretraining Decoder Language Model

2.1 Model and training setup

I implemented a GPT-like Transformer decoder from scratch with masked (causal) self-attention. As in Part 1, the model uses token embeddings plus absolute positional embeddings. Each decoder block

Epoch	Train Acc (%)	Test Acc (%)
1	47.90	40.27
2	53.73	44.93
3	64.10	56.80
4	71.37	60.80
5	76.77	65.47
6	84.18	72.67
7	89.63	77.20
8	88.86	76.13
9	95.41	83.60
10	96.37	81.73
11	97.66	83.73
12	96.18	83.20
13	98.04	83.07
14	98.80	85.33
15	98.80	84.93

Table 1: Part 1 accuracy after each epoch. Encoder parameters: 550,032.

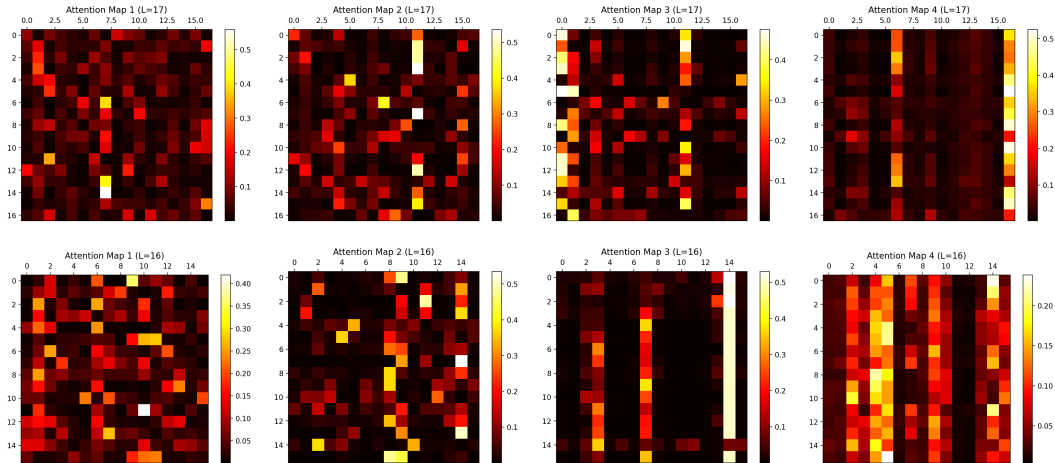


Figure 1: Part 1 attention heatmaps (2 sentences $\times$ 4 layers). Each panel shows the averaged attention map across heads for the non-padding token region.

applies causal masking to prevent attending to future tokens. The feedforward hidden dimensionality is 100 with ReLU activation, matching the assignment specification.

I pretrained the decoder on next-token prediction using the provided training text. Following the base setting, I trained for 500 iterations with batch size 16 and block size 32. The total number of decoder parameters is 985,142.

2.2 Sanity checks: attention normalization and visualization

I applied the same sanity-check procedure to the decoder. Figure 2 shows 8 attention panels (two sentences and four layers). The causal structure is visible: attention mass concentrates on the diagonal and lower triangle, consistent with masking out future tokens. This provides a direct visual confirmation that the model cannot attend to future tokens during autoregressive training.

2.3 Results: training perplexity checkpoints and test perplexity

Table 2 reports the training perplexity after every 100 iterations and at iteration 500, as required. Table 3 reports test perplexities on speeches by different politicians.

The test perplexities differ by politician, which is expected because each politician’s test set reflects a distinct distribution of lexical choice, phrasing, and rhetorical style. In these results, the decoder

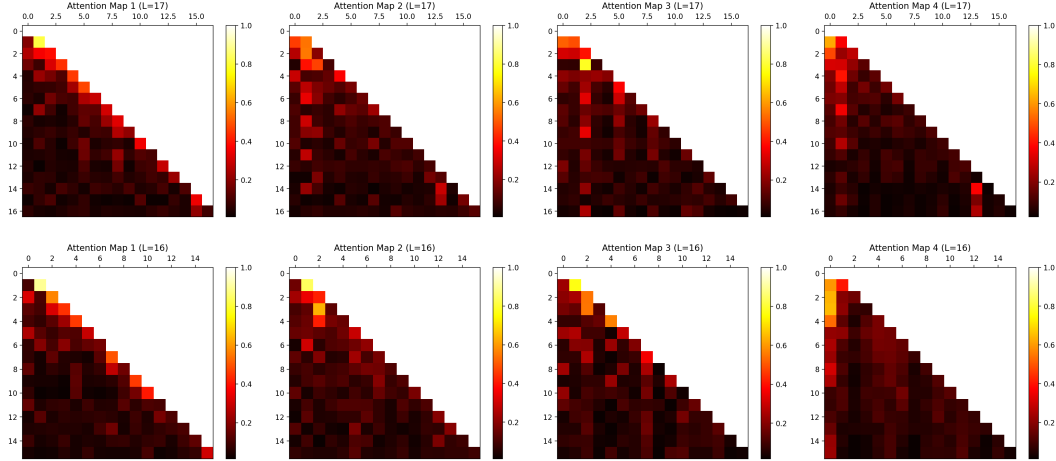


Figure 2: Part 2 decoder attention heatmaps (2 sentences $\times$ 4 layers). Causal masking yields a lower-triangular attention pattern, consistent with autoregressive decoding.

Iteration	Train Perplexity
100	753.50
200	657.24
300	485.64
400	345.65
500	250.02

Table 2: Part 2 training perplexity checkpoints for the decoder LM. Decoder parameters: 985,142.

achieves the lowest perplexity on Obama and the highest on G.W. Bush. One plausible explanation is that the training corpus distribution may be closer to Obama’s test distribution, or that Obama’s text is more lexically predictable under this tokenizer and small-context training regime. Another possibility is differing amounts of repetition and formulaic phrasing across speakers, which can reduce perplexity for a limited-capacity model.

3 Part 3: Architectural Exploration

3.1 Exploration choice: local-window sparse attention

For architectural exploration, I implemented local-window sparse attention in the decoder, inspired by sparse attention patterns used for efficient long-context modeling (2). Concretely, instead of allowing each token to attend to all previous tokens, I restricted attention to the most recent w tokens, where w is the window size. This reduces the effective attention bandwidth and can improve efficiency. It also provides a controlled way to evaluate how limiting context affects language modeling performance.

I evaluated two window sizes, $w = 8$ and $w = 16$, keeping all other hyperparameters identical to the Part 2 baseline.

3.2 Training dynamics and test perplexity comparisons

Table 4 reports training perplexity checkpoints for the two local-window settings. Table 5 reports test perplexities for the three politicians. In these runs, both windowed settings slightly improve test perplexity relative to the baseline, with $w = 8$ giving the largest improvement overall. A possible interpretation is that under this small-scale training regime (limited tokens and short block size), restricting attention can act as a structural regularizer by emphasizing recent context.

Test Set	Test Perplexity
Obama (test_LM_obama)	654.22
G.W. Bush (test_LM_wbush)	776.46
G.H. Bush (test_LM_hbush)	708.06

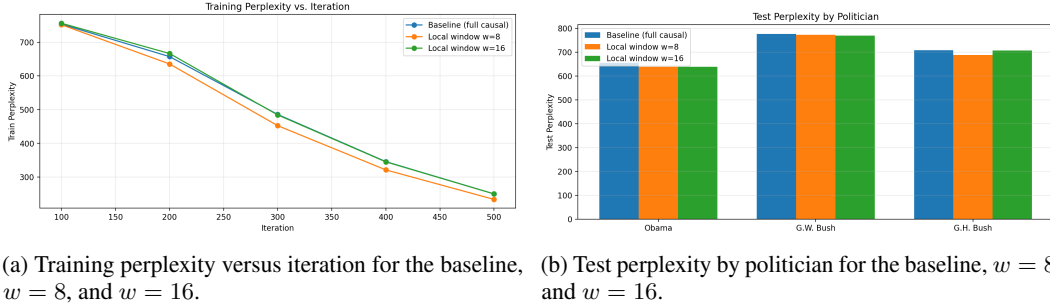
Table 3: Part 2 test perplexities for different politicians.

Setting	100	200	300	400	500
Local window $w = 8$	751.74	635.41	452.32	321.16	233.62
Local window $w = 16$	756.14	666.42	484.21	344.94	249.87

Table 4: Part 3 training perplexity checkpoints for local-window attention. Values are train perplexities at iterations 100/200/300/400/500.

Setting	Obama	G.W. Bush	G.H. Bush
Baseline (Part 2, full causal)	654.22	776.46	708.06
Local window $w = 8$	637.86	772.18	688.13
Local window $w = 16$	638.83	769.31	706.93

Table 5: Part 2 vs. Part 3 test perplexities across politicians. Local-window attention yields small improvements in these runs, with $w = 8$ providing the strongest gains.



(a) Training perplexity versus iteration for the baseline, $w = 8$, and $w = 16$. (b) Test perplexity by politician for the baseline, $w = 8$, and $w = 16$.

Figure 3: Part 3 visualization of training dynamics and downstream evaluation for dense causal attention (baseline) and local-window sparse attention ($w = 8$, $w = 16$).

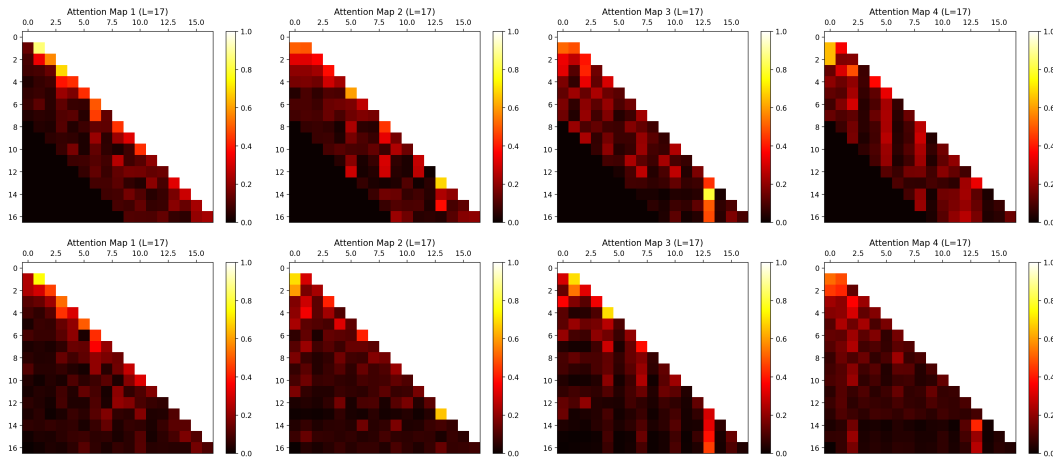


Figure 4: Part 3 decoder attention heatmaps for local-window attention ($w = 8$ vs. $w = 16$). The enforced locality should appear as a narrower attention band for $w = 8$ compared to $w = 16$.

3.3 Additional attention visualization for Part 3

To keep the narrative visually complete and consistent with Parts 1–2, I also include attention heatmaps for the local-window models. These panels help confirm that the intended sparse pattern is enforced, since the visible attention band should be confined to a fixed-width region near the diagonal.

4 Conclusion

In this assignment, I learned that implementing Transformer blocks from scratch is sufficient to obtain strong classification accuracy in a small supervised setting and reasonable perplexities in a small-scale autoregressive pretraining setup. The encoder-classifier reaches 84.93% test accuracy after 15 epochs, and the decoder achieves train perplexity 250.02 after 500 iterations with test perplexities in the 600–800 range across politicians. In architectural exploration, local-window sparse attention provides a simple inductive bias that improves perplexity in this limited-data regime, and the attention visualizations provide a direct sanity check that the sparse pattern is enforced.

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
- [2] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv:2004.05150*, 2020.
- [3] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *ICLR*, 2021.
- [4] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *ICLR*, 2022.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.
- [6] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- [7] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.
- [8] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv:1607.06450*, 2016.
- [9] Steven Bird. NLTK: The natural language toolkit. In *Proceedings of the ACL Interactive Poster and Demonstration Sessions*, 2006.
- [10] Andrej Karpathy. Neural Networks: Zero to Hero / Let’s build GPT from scratch. Online tutorial, 2023.